

301AA - Advanced Programming

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

AP-25: Python and the GIL

**and now, a few words on
Memory Management in....**

Python!!!



Garbage collection in Python

CPython manages memory with a **reference counting** + a **mark&sweep** cycle collector scheme

- **Reference counting**: each object has a counter storing the number of references to it. When it becomes 0, memory can be reclaimed.
- **Pros**: simple implementation, memory is reclaimed as soon as possible, no need to freeze execution passing control to a garbage collector
- **Cons**: additional memory needed for each object; cyclic structures in garbage cannot be identified (thus the need of **mark&sweep**)

Memory safety in Python

Getting the reference count:

```
import sys

list = [1, 3, 5]

rc = sys.getrefcount(list)

print(f"Ref count for my_list is: {rc}")
```

- No **dangling pointers** in Python
- No **double free** in Python

Race conditions in Python? NO

Example: Shared counter incremented 10k times in parallel by two threads.

```
# counter in closure
def counter_factory():
    counter = 0
    def counter_increaser():
        nonlocal counter
        counter = counter + 1
    return counter_increaser
```

```
# Runs fun() in parallel
def thread_fun(nthreads, fun):
    threads = []
    for _ in range(nthreads):
        threads.append(Thread(target = fun))
        threads[-1].start()
    for t in threads:
        t.join()
```

```
# decorator: repeats fun ntimes
def times(ntimes):
    """Usage:
    times(ntimes)(fun)(args,kwargs)"""
    def times_dec(fun):
        def wrapper(*args,**kwargs):
            for i in range(ntimes):
                fun(*args,**kwargs)
            return
        return wrapper
    return times_dec
```

```
inc = counter_factory()
thread_fun(2, times(10000)(inc))
inc.__closure__[0].cell_contents
```

Handling reference counters

- Updating the refcount of an object has to be done *atomically*
- In case of *multi-threading* you need to synchronize all the times you modify refcounts, or else you can have wrong values
- Synchronization primitives are quite expensive on contemporary hardware
- Since almost every operation in CPython can cause a refcount to change somewhere, handling refcounts with some kind of synchronization would cause *spending almost all the time on synchronization*
- As a consequence...

Concurrency in Python...



The Global Interpreter Lock (GIL)

- The CPython interpreter assures that only one native thread executes Python bytecodes at a time, thanks to the **Global Interpreter Lock**, which is a mutex on the Python interpreter
- The current thread must hold the **GIL** before it can safely access Python objects
- This simplifies the CPython implementation by making the object model (including critical built-in types such as **dict**) implicitly safe against concurrent access: no race conditions
- Locking the entire interpreter makes it easier for the interpreter to be multi-threaded, **at the expense of much of the parallelism afforded by multi-processor machines.**

More on the GIL

- However the GIL can degrade performance even when it is not a bottleneck. The system call overhead is significant, especially on multicore hardware.
- Two threads calling a function may take twice as much time as a single thread calling the function twice.
- The GIL can cause I/O-bound threads to be scheduled ahead of CPU-bound threads. And it prevents signals from being delivered.
- Some extension modules, either standard or third-party, are designed so as to release the GIL when doing computationally-intensive tasks such as compression or hashing.
- Also, the GIL is always released when doing I/O.

Alternatives to the GIL?

- Past efforts to create a “free-threaded” interpreter (one which locks shared data at a much finer granularity) have not been successful because performance suffered in the common single-processor case.
- It is believed that overcoming this performance issue would make the implementation much more complicated and therefore costlier to maintain.
- Guido van Rossum has said he will reject any proposal in this direction that slows down single-threaded programs.
- **Jython** (on JVM, -> 2017, Python 2.7) and **IronPython** (on .NET) have no GIL and can fully exploit multiprocessor systems
- **PyPy** (Python in Python, supporting JIT) currently has a GIL like CPython
- in **Cython** (compiled, for CPython extension modules) the GIL exists, but can be released temporarily using a "with" statement

Experimental feature: GIL optional in Python 3.13 (Oct. 2024)

- **Experimental Support for Free-Threaded Mode** where the GIL is disabled. This is aimed at improving multi-threading capabilities and enabling better performance in CPU-bound tasks.
- **Specializing Interpreter Enhancements**: The specializing interpreter (interpreter with some ad hoc optimizations) has undergone modifications to ensure thread safety without the GIL.
- **`sys._is_gil_enabled()`** to see if the current interpreter uses GIL
- The feature is potentially reversible if it breaks more of the current implementation of Cpython than expected.

Python 3.14 (7 Oct. 2025)

- **Support for Free-Threaded Mode** improved.
- Plan to be the default in future releases.